Université numérique
CHEIKH HAMIDOU KANE

Master Systèmes Réseaux et Infrastructures Virtuelles

Cours : Système Répartis

Projet : Conception et implémentation d'un système distribué de supervision réseau basé sur une architecture client–serveur.

Professeur : Dr. Maurice D. FAYE

Réalisé par : Abdelkerim Mahamat ZENI

N° INE ET114145241

GitHub : https://github.com/Abdel-13/Syst-me-repartis-projet.git

Sommaire

I- Introduction----------**3**

II- Architecture (schéma avec draw.io)----------**4**

III- Choix techniques (Python, type de pool)----------**5**

IV- Protocole de communication----------**6**

V- Résultats expérimentaux (test avec 100 clients)----------**7**

VI -Limites et améliorations----------**8**

VII - Conclusion et bibliographie----------**8**

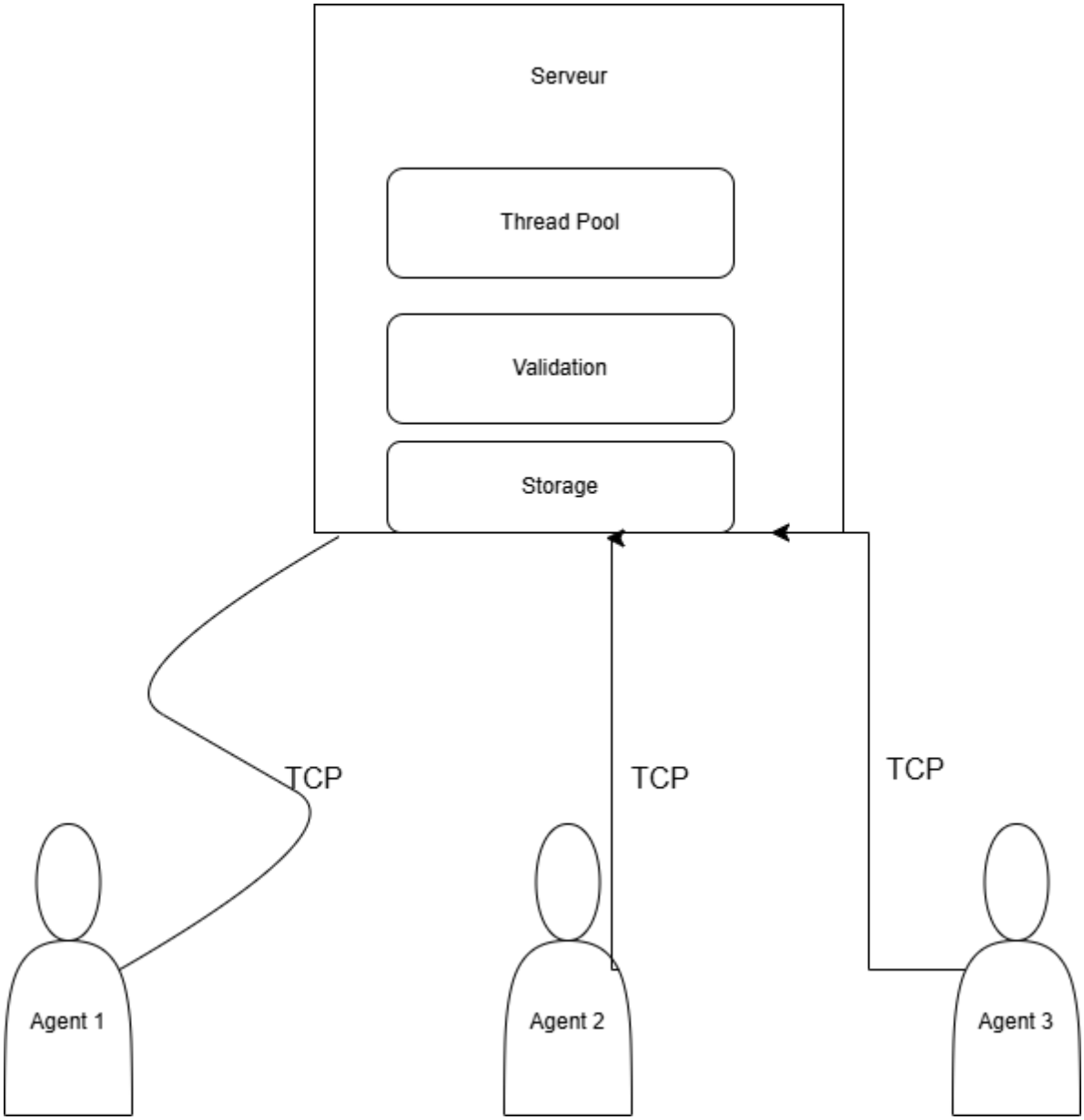
Introduction

Dans un monde où les infrastructures informatiques deviennent de plus en plus complexes et distribuées, la supervision des systèmes et des réseaux constitue un enjeu majeur pour garantir la disponibilité, la performance et la sécurité des services. Les organisations modernes s'appuient sur des solutions de monitoring telles que Nagios, Zabbix ou Prometheus afin de collecter, analyser et centraliser les métriques de leurs machines et applications.

Le présent projet réalisé dans le cadre du cours Systèmes Répartis du Master 1 SRIV à l'Université Numérique Cheikh Hamidou Kane, s'inscrit dans cette logique. Il consiste à concevoir et implémenter un système distribué de supervision réseau basé sur une architecture client-serveur. Chaque nœud supervisé exécute un agent chargé de collecter périodiquement des informations sur l'état du système (CPU, mémoire, disque, uptime, service et ports), puis de les transmettre au serveur central via une connexion TCP. Le serveur, multi-threadé et connecté à une base de données, réceptionne, valide et stocke ces métriques, tout en offrant une interface de consultation et de gestion.

Ainsi ce travail vise à fournir une architecture distribuée simple mais réaliste, permettant d'illustrer les principes fondamentaux de la supervision réseaux et de la programmation répartie.

I- Architecture (schéma avec draw.io)



Agent de supervision :

Pour créer l'agent de supervision en Python nous allons utiliser la bibliothèque psutil, qui est le Standard pour récupérer les métriques système.

L'agent doit collecter les données périodiquement et les envoyer au serveur via **une socket TCP**.

Prérequis :

Installons la bibliothèque nécessaire :

```
Administrateur : Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/P
s

PS C:\Windows\system32> pip install psutil
Collecting psutil
  Downloading psutil-7.2.2-cp37-abi3-win_amd64.whl.metadata (22 kB)
Downloading psutil-7.2.2-cp37-abi3-win_amd64.whl (137 kB)
   ---------------------------------------- 137.7/137.7 kB 52.3 kB/s eta 0:00:00
Installing collected packages: psutil
Successfully installed psutil-7.2.2

[notice] A new release of pip is available: 24.0 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Windows\system32>
```

III- Choix techniques (Python, type de pool et SQLite)

L'importance de choix techniques réside dans la stabilité et la scalabilité du Système de supervision.

1 - Python :

Avec ses bibliothèques spécialisées on arrive on arrive à récupérer des métriques systèmes complexes (CPU, RAM…) en une seule ligne de code.

2 - Le Pool de Threads :

- Gestion des ressources :La création d'un nouveau thread pour chaque agent consomme beaucoup de mémoire. Le Pool limite le nombre de threads actifs, protégeant le serveur contre un crash si 100 agents se connectent en même temps.
- Réactivité : Les threads sont déjà créés et attendent de la tâche, ce qui réduit le temps de la réponse.

3 – Base de données au Pool de connexion :

- Sécurité SQLite : ça permet de contrôler l'accès des threads.
- Performance : La réutilisation de connexions existants accélère considérablement l'enregistrement des métriques.

IV- Protocole de communication

Le protocole de communication est le langage et l'ensemble de règles qui permettent aux Agents et le Serveur de se comprendre, il se décompose en trois couches principales :

1-Transport :TCP(Transmission Control Protocol) est crucial pour la fiabilité :

- Mode connecté :Une connexion est établie avant tout échange, garantissant que le Serveur est prêt à recevoir.
- Contrôle d'erreur : TCP garantit que chaque paquet de métrique arrive dans le bon ordre et sans perte.
- Adressage : Utilisation du couple **IP/Port (127.0.0.1 :5000)**

La Commande netstat sert à identifier quel programme utilise le port **5000**

netstat -ano | findstr :5000

```
PS C:\Windows\system32> netstat -ano | findstr :5000
  TCP    127.0.0.1:5000         0.0.0.0:0              LISTENING       11476
PS C:\Windows\system32>
```

Pour libérer le port on utilise :

taskkill /F /PID Id(11476)

```
PS C:\Windows\system32> taskkill /F /PID 11476
Opération réussie : le processus avec PID 11476 a été terminé.
PS C:\Windows\system32>
```

2- Format des données : JSON (JavaScript Object Notation) sans ce format de données l'agent et le serveur ne pourraient pas communiquer. JSON c'est un format texte léger facile à lire et très simple en python, les données sont envoyées sous formes de paires « Clé-Valeur »

3- Modèle d'échange : Le système utilise un modèle poussé d'informations « PUSH » Initié par le client : l'agent se connecte toutes les 10 secondes.

V- Résultats expérimentaux (test avec 100 clients)

1 – Test :

- Environnement :Machine locale(Localhost) ;
- Outil : Script Python lançant 100 instances d'agents en boucle ;
- Fréquence :Chaque agent envoie des données toutes les 5 secondes ;
- Volume : Environ 20 messages reçus par seconde par le serveur.

- **Connexions des agents de 1 à 100.**

```
(.venv) PS C:\Users\Abdel\PycharmProjects\Systèmes répartis
> python agents.py connexion
--- Connexion ordonné de 100 agents (1 à 100) ---
Connexion de Agent-001...
Connexion de Agent-002...
Connexion de Agent-003...
Connexion de Agent-004...
Connexion de Agent-005...
Connexion de Agent-006...
Connexion de Agent-007...
Connexion de Agent-008...
Connexion de Agent-009...
Connexion de Agent-010...
Connexion de Agent-011...
Connexion de Agent-012...
Connexion de Agent-013...
Connexion de Agent-014...
Connexion de Agent-015...
Connexion de Agent-016...
Connexion de Agent-017...
Connexion de Agent-018...
```

```
Connexion de Agent-080...
Connexion de Agent-081...
Connexion de Agent-082...
Connexion de Agent-083...
Connexion de Agent-084...
Connexion de Agent-085...
Connexion de Agent-086...
Connexion de Agent-087...
Connexion de Agent-088...
Connexion de Agent-089...
Connexion de Agent-090...
Connexion de Agent-091...
Connexion de Agent-092...
Connexion de Agent-093...
Connexion de Agent-094...
Connexion de Agent-095...
Connexion de Agent-096...
Connexion de Agent-097...
Connexion de Agent-098...
Connexion de Agent-099...
Connexion de Agent-100...

Terminé : Les 100 agents tournent en arrière-plan.
(.venv) PS C:\Users\Abdel\PycharmProjects\Systèmes
```

- Enregistrement de données Côté Serveur.

```
"C:\Users\Abdel\PycharmProjects\Systèmes répartis\.venv\Scripts\python.
SERVEUR ACTIF sur 127.0.0.1:5000...
[2026-03-12 16:22:36] Agent-002 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:37] Agent-003 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:37] Agent-001 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:37] Agent-004 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:37] Agent-005 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:37] Agent-006 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:38] Agent-007 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:38] Agent-008 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:38] Agent-009 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:38] Agent-010 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:39] Agent-011 connecté | CPU: 100.0% | RAM: 49.4%
[2026-03-12 16:22:39] Agent-013 connecté | CPU: 100.0% | RAM: 49.5%
[2026-03-12 16:22:39] Agent-014 connecté | CPU: 100.0% | RAM: 49.5%
[2026-03-12 16:22:39] Agent-012 connecté | CPU: 100.0% | RAM: 49.5%
[2026-03-12 16:22:40] Agent-015 connecté | CPU: 100.0% | RAM: 49.5%
[2026-03-12 16:22:40] Agent-016 connecté | CPU: 100.0% | RAM: 49.5%
[2026-03-12 16:22:40] Agent-017 connecté | CPU: 100.0% | RAM: 49.5%
[2026-03-12 16:22:41] Agent-018 connecté | CPU: 100.0% | RAM: 49.6%
[2026-03-12 16:22:41] Agent-019 connecté | CPU: 100.0% | RAM: 49.6%
[2026-03-12 16:22:41] Agent-020 connecté | CPU: 100.0% | RAM: 49.6%
```

VI -Limites et améliorations

Le système actuel est limité par la gestion des accès concurrents de SQLite et l'absence de chiffrement des données circulant sur le réseau. Pour passer à une échelle organisationnelle, il serait nécessaire de migrer vers une base de données PostgreSQL et d'implémenter le protocole SSL/TLS. L'ajout d'une interface web(Flash/Django) et d'un système d'alerte automatique par mail permettrait une surveillance anticipatrice et ergonomique.

VII - Conclusion et bibliographie

La réalisation de ce projet de supervision distribuée a permis de concevoir une architecture logicielle robuste capable de monitorer en temps réel un parc informatique simulé de **100 machines.** L'utilisation des sockets **TCP/IP** en Python, couplée à une gestion de base de données **SQLite** structurée, a démontré l'efficacité de la communication Client-Serveur pour la collecte des données.

Ce travail m'a permis de relever des défis techniques majeurs, notamment la gestion de la **concurrence** via le multi-threading et la synchronisation des données lors de tests de charges massives.

1- Bibliographie

Ouvrages et supports de cours

- Documentation officielle Python 3.12 : Socket Programming HOWTO et sqlite3 module guide
- Supports de cours « Systèmes Répartis » : Concepts de client-serveur

Ressources numériques :

- Draw IO : Pour la création de diagramme de connexions de agents. https://app.diagrams.net
- PSUTIL Documentation : Cross-platform lib for process and system monitoring in Python. https://psutil.readthedocs.io
- SQLite Browser Officiel : Guide d'utilisation de l'interface graphique pour la gestion de bases de données. https://sqlitebrowser.org
- Real Python : Socket Programming in Python (Guide).